Constructors in Java

Constructors in Java

1. Instance Variables vs. Local Variables
2. What Is a Constructor?
3. The Default Constructor
4. Parameterized Constructors & Constructor Overloading
5. The `this` Keyword
6. Constructor Chaining with `this()`
7. Rules and Restrictions Around Constructor Calls
🌍 Real-Life Examples of Constructors in Practice
⭐ One-Page Revision
📝 Practice Questions
🔥 Quick FAQ

Constructors in Java

Default Values, Overloading, the `this` Keyword, and Constructor Chaining

1. Instance Variables vs. Local Variables

```
Student s1 = new Student();
System.out.println(s1.name);        // null
System.out.println(s1.age);         // 0
System.out.println(s1.rollNumber);  // 0
System.out.println(s1.college);     // null
```

Even without assigning a single value to `s1`, this code runs **without errors** and prints something. This works because of **default values** — but only for a specific category of variable.

	Instance Variable	Local Variable
Declared	Inside a class, outside any method (like `name`, `age` in `Student`)	Inside a method or block (like `int x = 4;` inside `main`)
Stored in	**Heap** (as part of the object)	**Stack**
Gets a default value?	✅ Yes, automatically	❌ No
Using it without assigning a value	Fine — prints its default value	**Compile-time error**

Definition: Instance variables are the fields declared inside a class that represent an object's characteristics. Since every object lives in the heap, its instance variables live there too — and Java automatically initializes them to a **default value** the moment the object is created.

Default Values by Data Type

Data Type	Default Value
Integer types (`int` , `byte` , `short` , `long`)	`0`
Floating-point (`float` , `double`)	`0.0`
`boolean`	`false`
`String` (and all non-primitive/reference types)	`null`

📌 **Important:** `null` means **"nothing"** — it's Java's way of saying a reference variable currently doesn't point to any actual object. It's not the same as an empty string `""`; Java specifically chose `null` because reference variables store *addresses*, and "no address yet" is naturally represented as null rather than zero.

⚠ **Common Mistake:** Trying this same trick with a **local** variable fails:

```
        int x;
        System.out.println(x);   // Compile-time error: variable x might not have been
initialized
```

Local variables get **no** default value — Java forces you to explicitly assign one before use.

💡 **Easy Hinglish Explanation**

> "Instance variables (jo class ke andar, method ke bahar declare hote hain) ko automatically default value mil jaati hai — kyunki wo object ka part hain, aur object heap mein banta hai. Lekin local variables (jo kisi method ke andar declare hote hain) ko koi default value nahi milti — unhe explicitly value deni hi padegi, warna compiler error aayega."

---

2. What Is a Constructor?

Manually assigning every field after creating an object is tedious and risky — forget one assignment, and you're left with an incompletely initialized object (silently holding default values instead of real data). **Constructors** solve this by initializing an object automatically, right when it's created.

> **Definition:** A constructor is a special block of code, written inside a class, that runs **automatically** the moment an object is created — used specifically to initialize that object's instance variables.

The 5 Rules of a Constructor

1. **Same name as the class.** A constructor for `Student` must be named exactly `Student`.
2. **No return type — not even `void`.** Constructors never declare a return type, unlike every other method.
3. **Automatically called during object creation.** You never call it directly by name like a normal method.
4. **Used to initialize an object.** Its entire purpose is setting up instance variables.
5. **It can be overloaded.** A class can have multiple constructors, differing in parameters — exactly like method overloading.

```
class Student {
    String name;
    int age;
    int rollNum;
    String college;

    Student() {
        name = "Aditya";
        age = 28;
        rollNum = 101;
        college = "IIT G";
    }
}
```

⭐ **Exam Tip:** When you write `Student s1 = new Student();`, that trailing `()` after `Student` isn't decorative — it's a genuine **method call**, calling the constructor. `new` allocates the heap memory; the `Student()` part is what actually invokes the constructor to initialize it.

---

3. The Default Constructor

What happens if you **never** write a constructor yourself, yet `new Student()` still works?

> 📌 **Important:** Java treats a constructor as **mandatory**. If you don't write one, Java silently generates an empty one for you at compile time, called the **default constructor**.

```
Student() {
    // completely empty — auto-generated by Java
}
```

This is exactly why the earlier example (Section 1) compiled fine and printed default values — Java's auto-generated default constructor did nothing, so every instance variable was left holding its default value.

⚠ **Common Mistake — a critical rule to remember:** The moment you write **any** constructor yourself (even a no-argument one), Java **stops** generating its own default constructor. If your code then tries to call `new Student()` with no arguments, and your only defined constructor requires parameters, you'll get a **compile-time error**: *"the constructor Student() is undefined."*

```
class Student {
    String name; int age; int rollNumber; String college;
    Student(String n, int a, int rn, String c) {   // you defined ONE constructor
        name = n; age = a; rollNumber = rn; college = c;
    }
}
// Student s2 = new Student();   ❌ Compile error — no matching (no-arg) constructor exists
```

---

4. Parameterized Constructors & Constructor Overloading

A **parameterized constructor** accepts arguments, letting you set an object's data dynamically at creation time instead of hardcoding fixed values:

```
        class Student {
            String name; int age; int rollNumber; String college;

            Student() { }   // default constructor, explicitly written

            Student(String n, int a, int rn, String c) {   // parameterized constructor
                name = n;
                age = a;
                rollNumber = rn;
                college = c;
            }
        }
```

Both constructors can now be used — this is **constructor overloading**, the same rules as regular method overloading (same name, different parameter lists):

```
        Student s1 = new Student("Rohit", 28, 102, "IIT G");   // calls parameterized
constructor
        Student s2 = new Student();                             // calls the no-arg
constructor
```

✅ **Best Practice:** Writing your own empty no-arg constructor **alongside** a parameterized one is a common, deliberate pattern — it restores the flexibility of "create with no data yet" that you'd otherwise lose the moment you add a parameterized constructor.

---

5. The `this` Keyword

Once a constructor's **parameter names** match the class's **instance variable names** (a very natural thing to do), a genuine ambiguity appears:

```
        Student(String name, int age, int rollNumber, String college) {
            name = name;   // ❌ Ambiguous — which "name"? Refers to the parameter, not
the field!
        }
```

Here, `name = name` doesn't do what you'd expect — both sides refer to the **parameter**, since it's the closest matching variable in scope. The instance variable never actually gets assigned.

> **Definition:** `this` is a special keyword that always refers to the **current object** — the specific object whose constructor or method is currently executing. `this.fieldName` unambiguously means "the instance variable belonging to this exact object," resolving the naming clash.

```
        Student(String name, int age, int rollNumber, String college) {
            this.name = name;             // this.name = instance variable, name =
parameter
            this.age = age;
            this.rollNumber = rollNumber;
            this.college = college;
        }
```

💡 Easy Hinglish Explanation

> "Jab constructor ke parameter ka naam bilkul instance variable jaisa hi ho, to Java confuse ho jaata hai ki hum kis 'name' ki baat kar rahe hain. `this.name` likhne se Java ko pata chal jaata hai ki hum current object ke instance variable ki baat kar rahe hain, na ki parameter ki."

---

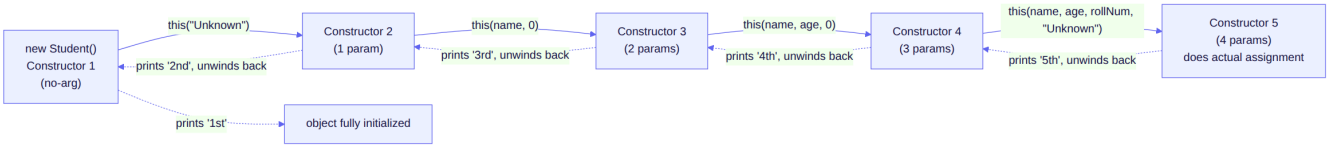
6. Constructor Chaining with `this()`

When multiple overloaded constructors exist, each one might otherwise repeat similar assignment logic. **Constructor chaining** lets one constructor call *another* constructor in the same class, avoiding this duplication.

```
class Student {
    String name; int age; int rollNumber; String college;

    Student() {
        this("Unknown");
        System.out.println("I am in first constructor");
    }
    Student(String name) {
        this(name, 0);
        System.out.println("I am in second constructor");
    }
    Student(String name, int age) {
        this(name, age, 0);
        System.out.println("I am in third constructor");
    }
    Student(String name, int age, int rollNumber) {
        this(name, age, rollNumber, "Unknown");
        System.out.println("I am in fourth constructor");
    }
    Student(String name, int age, int rollNumber, String college) {
        this.name = name;
        this.age = age;
        this.rollNumber = rollNumber;
        this.college = college;
        System.out.println("I am in fifth constructor");
    }
}
```

Calling `new Student()` triggers a chain reaction: the no-arg constructor calls the 1-parameter one, which calls the 2-parameter one, which calls the 3-parameter one, which finally calls the 4-parameter constructor — the only one that does the **actual field assignment**.



Execution order: Notice the `println` statements execute in **reverse** of the calling order — `this(...)` is like a function call that must fully complete (and return) before the line *after* it runs. So the chain calls all the way down to the 5th constructor first, and only *after* it finishes does control unwind back up through 4th → 3rd → 2nd → 1st, printing each message on the way back.

Actual output of `new Student()`:

```
I am in fifth constructor
I am in fourth constructor
I am in third constructor
I am in second constructor
I am in first constructor
```

Followed by the final field values: `Unknown 0 0 Unknown` (from `s1.name`, `s1.age`, `s1.rollNumber`, `s1.college`).

⭐ **Exam Tip:** This chaining behavior directly mirrors **recursion** — each constructor "calls deeper" before doing its own final work, then unwinds back out, printing on the way back up — the exact same call-stack pattern as a recursive function.

The Critical Rule: `this()` Must Be the First Statement

⚠ **Common Mistake:** Writing anything **before** the `this(...)` call inside a constructor causes a compile error. Java strictly requires `this(...)` to be the **very first line** of the constructor:

```
        Student() {
            System.out.println("I am in first constructor");  // ❌ written before this() — ERROR
            this("Unknown");
        }
```

```
        Student() {
            this("Unknown");                                  // ✅ correct — this() is first
            System.out.println("I am in first constructor");
        }
```

---

7. Rules and Restrictions Around Constructor Calls

⚠ **Interview-favorite question:** *Can a constructor be called manually, like a regular method?*

No. A constructor can only be invoked in exactly **two** situations:

1. Automatically, the first time an object is created with `new`
2. Via `this(...)`, from inside another constructor of the same class (chaining)

```
        Student s1 = new Student();   // ✅ valid — implicit call during object creation
        // Student.Student();          ❌ not valid — cannot call a constructor like a normal method
```

Runtime Exception: Insufficient Heap Space

Another common interview point: since `new` allocates memory **dynamically, at runtime**, it's technically possible for the heap to already be full when an object is created.

> 📌 **Important:** If there isn't enough space left in heap memory to allocate a new object, Java throws a **runtime exception** — this can't be caught at compile time, since heap availability is only known while the program is actually running.

💡 **Easy Hinglish Explanation**

> "`new` keyword heap memory ko runtime pe allocate karta hai — matlab jab program chal raha ho, tabhi. Agar us waqt heap memory already bhari hui ho, to naye object ke liye jagah nahi milegi, aur ek runtime exception aa jaayegi. Ye compile time pe pakड़ी nahi ja sakti, kyunki heap ki available space sirf runtime pe pata chalti hai."

🌍 Real-Life Examples of Constructors in Practice

- **Signing up for an app** — when you create an account, a `User` object is constructed with your name, email, and a default `isVerified = false`, exactly like `Student()`'s default values before you fill in real data.
- **Placing an online order** — an `Order` object's parameterized constructor takes the item list, address, and payment method all at once, guaranteeing an order is never created half-filled — the same reason `Student(name, age, rollNumber, college)` forces all four values together.
- **Configuring a car in a dealership app** — constructor overloading lets you create a `Car` with just a model name (defaults for everything else) or with a fully specified trim, color, and engine — same pattern as the `Student()` vs. `Student(name, age, rollNum, college)` overloads.
- **Building a settings object in games** — constructor chaining is used so a "quick start" constructor (few parameters) can call into a "full custom" constructor (all parameters), avoiding rewriting the same setup logic twice.

⭐ One-Page Revision

Core Definitions

- **Instance variable** = a field declared in a class, stored in the heap as part of an object; gets an automatic default value
- **Local variable** = a variable declared inside a method, stored in the stack; gets **no** default value
- **Constructor** = a special block that auto-runs on object creation, used to initialize instance variables
- **Default constructor** = the empty, no-argument constructor Java auto-generates *only if* you define no constructor yourself
- `this` = refers to the current object; resolves naming clashes between parameters and instance variables
- **Constructor chaining** = one constructor calling another via `this(...)`, avoiding duplicated initialization code

Default Values Cheat Sheet | Type | Default | |---|---| | int/byte/short/long | 0 | | float/double | 0.0 | | boolean | false | | String / any object reference | null |

Key Rules

- Constructor name = class name; no return type; auto-invoked; can be overloaded
- Writing **any** constructor yourself removes Java's auto-generated default constructor
- `this(...)` must always be the **first line** in a constructor
- Constructors can **never** be called manually like a regular method — only via `new` or `this(...)`
- Insufficient heap space during `new` throws a **runtime exception**, not a compile-time error

---

📝 Practice Questions

Basic

1. What are the 5 rules a constructor must follow?
2. What is the default value of an uninitialized `int` instance variable? What about a `String`?

3. Why does `int x; System.out.println(x);` fail to compile, but the same pattern works fine for instance variables?

Medium 4. Write a `Book` class with fields `title`, `author`, `price`, a no-arg constructor, and a parameterized constructor. Create one object with each. 5. Why does defining a parameterized constructor yourself remove Java's default constructor? What error appears if you then try `new Student()` with no matching constructor? 6. In a chain of 3 constructors using `this(...)`, which constructor's `println` statement executes *first* at runtime — the first one called, or the last one in the chain? Why?

Interview 7. Can you call a constructor manually, the way you'd call `someMethod()`? Justify your answer with the two valid ways a constructor *can* be invoked. 8. Why must `this(...)` always be the first statement in a constructor? 9. Explain how a heap-space runtime exception could occur during object creation, and why it can't be detected at compile time. 10. Rewrite this ambiguous constructor correctly using `this`: `java Student(String name) { name = name;   // fix this line }`

🔥 Quick FAQ

If I don't write any constructor, is my object still valid? Yes — Java's auto-generated default constructor still runs, just leaving every instance variable at its default value (`0`, `false`, or `null`).

Can a class have both a default and a parameterized constructor? Yes — this is exactly constructor overloading, and it's a very common, deliberate pattern.

Does `this` only work inside constructors? No — `this.fieldName` also works inside any instance method to resolve the same kind of naming clash with a method's parameters, not just constructors.

Is constructor chaining mandatory? No — it's an optional technique to avoid duplicating initialization logic across overloaded constructors. Each constructor could still fully initialize fields independently without chaining.